COSC 419/519B – Deep Learning
Winter 2025 T2
Dr. M. S. Shehata

A3 (20 points) – Due Feb. 23rd, 2026

Focus: PyTorch, CNN

1. [4 points] Setting Up PyTorch and Preparing the Data
   1.1. **Importing Libraries:** Import torch, torch.nn, torch.optim, torchvision, torchvision.transforms, matplotlib.pyplot, numpy, and tqdm.auto. Set a manual seed of 42 for reproducibility. Also, add code to check and print the device being used (CPU or GPU).
   1.2. **Downloading and Loading the FashionMNIST Dataset:** We'll use the torchvision library to download and load the FashionMNIST dataset. We'll also apply some transformations to the data:
      1. ToTensor: Convert the images to PyTorch tensors.
      2. Normalize: Normalize the pixel values to the range [-1, 1]. Since FashionMNIST is grayscale, we only need one value for the mean and standard deviation.

      We'll create data loaders for both the training and testing sets, using a batch size of 32 and 2 worker processes.
   1.3. **Visualizing the Data:** Let's visualize a few images from the training set to get a feel for the data. We'll create a grid of images manually using matplotlib.pyplot.subplots to have more control over the layout.
      1. Get a batch of training images and their corresponding labels.
      2. Determine the number of rows and columns for the grid (e.g., 8 images per row).
      3. Create a figure and a grid of subplots using plt.subplots().
      4. Iterate through the images:
         a. Unnormalize each image.
         b. Plot the image on the corresponding subplot using ax.imshow().
         c. Set the label as the title of the subplot using ax.set_title(), ensuring the label is clearly visible on top of the image.
         d. Turn off the axes for each subplot using ax.axis('off').
      5. Remove any unused subplots if the number of images is not a multiple of the number of columns.
      6. Adjust the spacing between subplots using plt.subplots_adjust() if needed.
      7. Display the plot using plt.show().

      Make sure the label for each image is clearly displayed on top of the image within the grid, and the overall layout is neat and organized.

2. [2 points] Building the Model
   2.1. **Define the VGG16-like CNN Class:** Create a class VGG16LikeNet that inherits from nn.Module. Define the layers as described. This architecture is a simplified VGG16, adapted for FashionMNIST's 28x28 input size.

      Architecture:

      The network has several convolutional blocks followed by fully connected layers:

      Block 1:

Conv2d(1, 64, kernel_size=3, padding=1)

ReLU()

Conv2d(64, 64, kernel_size=3, padding=1)

ReLU()

MaxPool2d(kernel_size=2, stride=2)

Block 2:

Conv2d(64, 128, kernel_size=3, padding=1)

ReLU()

Conv2d(128, 128, kernel_size=3, padding=1)

ReLU()

MaxPool2d(kernel_size=2, stride=2)

Block 3:

Conv2d(128, 256, kernel_size=3, padding=1)

ReLU()

Conv2d(256, 256, kernel_size=3, padding=1)

ReLU()

Conv2d(256, 256, kernel_size=3, padding=1)

ReLU()

MaxPool2d(kernel_size=2, stride=2)

Block 4:

Conv2d(256, 512, kernel_size=3, padding=1)

ReLU()

Conv2d(512, 512, kernel_size=3, padding=1)

ReLU()

Conv2d(512, 512, kernel_size=3, padding=1)

ReLU()

MaxPool2d(kernel_size=2, stride=2) Note: No pooling after Block 4

Block 5:

Conv2d(512, 512, kernel_size=3, padding=1)

ReLU()

Conv2d(512, 512, kernel_size=3, padding=1)

ReLU()

Conv2d(512, 512, kernel_size=3, padding=1)

ReLU()

MaxPool2d(kernel_size=2, stride=2)

Fully Connected Layers:

Linear(512 * 1 * 1, 4096) (Input size depends on previous layers. FashionMNIST is 28x28; five 2x2 max-pooling layers result in a 1x1 feature map. We have 512 channels.)

ReLU()

Linear(4096, 4096)

ReLU()

Linear(4096, 10) (10 output classes)

Important: Initialization (Poor Initialization)

1. Create an init_weights_poor function:
    - Take a module m as input.
    - Check m's type with isinstance:
        - If m is nn.Conv2d or nn.Linear:
            i. Initialize weights with a normal distribution (mean 0, std 0.1): m.weight.data.normal_(0, 0.1)
            ii. Initialize bias to 0: m.bias.data.fill_(0.0) if m.bias is not None.
2. Apply Initialization: After defining all layers in __init__, apply:

   self.apply(init_weights_poor)

2.2. **Instantiate the Model and Print its Architecture:** Create an instance of the VGG16LikeNet class and call it net. Print the model's architecture using print(net). **Important:** Move the model to the GPU using net = net.to(device).

3. [3 points] Training the Model (with Poor Initialization)

3.1. **Define Loss Function and Optimizer:**
- Define the loss function as nn.CrossEntropyLoss() and call it criterion.
- Define the optimizer as optim.Adam with a learning rate of 0.001, and pass net.parameters() to it. Call the optimizer optimizer.

3.2. **Training Loop**: Implement the training loop

1. Set the number of epochs to a small value initially (e.g., 5 or 10) to observe the initial training behavior. We'll increase it later.
2. Iterate over the training data for each epoch. Use `tqdm` to wrap the epoch loop for a progress bar.
3. In each iteration:
    a. Get the inputs and labels from the data loader and move them to the GPU using `.to(device)`.
    b. Zero the parameter gradients using `optimizer.zero_grad()`.
    c. Perform the forward pass: `outputs = net(inputs)`.
    d. Compute the loss: `loss = criterion(outputs, labels)`.
    e. Perform the backward pass: `loss.backward()`.
    f. Update the parameters: `optimizer.step()`.
    g. Accumulate the loss.
4. Print the average loss for each epoch.
5. Store the training loss after each epoch in a list called `losses`.
6. After training, plot the losses using `matplotlib`.

4. [3 points] Testing the Model (with Poor Initialization)

4.1. **Evaluate the Model on the Test Set:** Compute the accuracy of the trained model on the entire test set.

1. Iterate through the test data using the testloader.
2. For each batch:
    - Get the inputs and labels and move them to the GPU.
    - Perform a forward pass to get the model's predictions: outputs = net(inputs).
    - Get the predicted class labels using torch.max(outputs, 1).
    - Update the total number of samples and the number of correctly classified samples.
3. Calculate the overall accuracy: accuracy = 100 * correct / total.
4. Print the accuracy.

4.2. **Evaluate Class-wise Accuracy:**

Now, let's see how the model performs on each class individually. Calculate and print the accuracy for each class.

1. Create a list to store the number of correctly classified samples for each class class_correct.
2. Create a list to store the total number of samples for each class class_total.
3. Iterate over the test data.
    - Move data to GPU.
4. For each batch update the class_correct and class_total.
5. Print the accuracy for each class.

4.3 Visualizing Failure Cases:

We'll visualize some of the incorrect predictions. This will help you see *which* types of images are confusing the poorly-initialized network. We can reuse and slightly modify the code of the previous visualization.

1. Create empty lists, wrong_images, wrong_labels, and wrong_predictions to store the failure cases.

2. Iterate through the test data using the testloader.
3. For each batch:
    - Get the inputs and labels and move them to the GPU using .to(device).
    - Perform a forward pass to get the model's predictions: outputs = net(inputs).
    - Get the predicted class labels using torch.max(outputs, 1).
    - Compare the predicted labels with the ground truth labels.
    - If a prediction is incorrect, append the image, ground truth label, and predicted label to the respective lists (wrong_images, wrong_labels, wrong_predictions).
4. After iterating through the entire test set, display a subset of the failure cases (e.g., the first 32) in a grid:
    - Determine the number of rows and columns for the grid (similar to Task 1.3).
    - Create a figure and a grid of subplots using plt.subplots().
    - Iterate through the collected failure cases (up to the maximum number you want to display):
        - Unnormalize each image and move to CPU for display.
        - Plot the image on the corresponding subplot using ax.imshow().
        - Create a combined label string showing both the ground truth (GT) and predicted (Pred) labels.
        - Set the combined label as the title of the subplot using ax.set_title(), ensuring it's clearly visible.
        - Turn off the axes for each subplot using ax.axis('off').
    - Remove any unused subplots.
    - Adjust the spacing between subplots using plt.subplots_adjust() if needed.
    - Display the plot using plt.show().

Hint: You can adapt the grid layout code from Task 1.3. Focus on clearly displaying the image, the ground truth label, and the incorrect prediction for each failure case.

5. [10 points] Improving the Model with He Initialization
   5.1. **Understand He Initialization**

Read the Paper (Optional):

"Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification" by He et al. (2015): https://arxiv.org/abs/1502.01852

5.2. **Implementing He Initialization (with an Initialization Function)**

1. Create a New Class: Create a new class VGG16LikeNetHe that inherits from nn.Module.
2. Copy Architecture: Copy the __init__ and forward methods from your original VGG16LikeNet class into VGG16LikeNetHe. Do not include any initialization within the __init__ method itself (besides defining the layers).
3. Create an init_weights function:
   - Name the function init_weights.
   - Inside init_weights, use isinstance to check the module type (m):
     - If m is an nn.Conv2d layer:
       a. Calculate fan_in: This is the number of input channels multiplied by the kernel height and width.
       b. Calculate the standard deviation (std): std = sqrt(2 / fan_in). Note: Only fan_in is used.
       c. Initialize the weights using m.weight.data.normal_(0, std). This directly draws from a normal distribution with mean 0 and the calculated std.
       d. If the layer has a bias (it usually does), initialize it to zero: m.bias.data.zero_() if m.bias is not None.
     - If m is an nn.Linear layer:
       a. Calculate fan_in: This is simply the number of input features.
       b. Calculate the standard deviation (std): std = sqrt(2 / fan_in). Note: Only fan_in is used.
       c. Initialize the weights: m.weight.data.normal_(0, std).
       d. If the layer has a bias, Initialize it to zero: m.bias.data.zero_() if m.bias is not None.
4. Apply Initialization: After defining the layers inside the __init__ method of VGG16LikeNetHe, apply the initialization: self.apply(init_weights).

5.3. **Instantiate, Train, and Evaluate:** Instantiate, train, and evaluate the model with He initialization.
   1. **Instantiate:** net_he = VGG16LikeNetHe(). Move it to the GPU.
   2. **Train:** Use the training loop from Section 3.2 (with net_he, a new optimizer optimizer_he, and a new loss list losses_he).
   3. **Evaluate:** Use the evaluation code from Sections 4.1 and 4.2 (with net_he).

Comparing Plots: Compare the training losses of the poorly initialized model and the He-initialized model. Hint: take the log of the losses since the scale difference is likely big.

Submission Instructions

1- Each question is linked to a cell in the provided notebook, provide your code whenever you see a comment "Your code here". (no group work).
2- Create text cells to answer the theoretical questions whenever necessary.
3- Submit the notebook to Canvas. Make sure your whole notebook runs without errors. Note that you can resubmit an assignment, but the new submission overwrites the old submission and receives a new timestamp.